

Intro

Developed an Data Analysis agent based on ReAct (Reasoning + Acting) paradigm capable of interpreting analytics queries on tabular datasets, generate and run python code in a secure sandbox, producing interactive Plotly views in HTML and tracking the reasoning trace for observability and transparency.

I'm using an Exhagonal Architecture (ports + adapters). Independent agent logic core (domain + use cases) without connection with the outside: no FastAPI, no specific DBs, nor LLM vendor have interaction with the core.

System architecture

External adapters layer:

- Fast API adapter (/analyze, /sessions, /plots)
- Open AI adapter (openai, gemini, ollama, ...)
- Mock LLM adapter (deterministic testing)
- Process Sandbox adapter (timeout, tempDir, socket-blocking)
- SQLite session repository (for the 2° day task)

Use cases Layer

- Analyze dataset use case (analytic flow orchestration)

Core domain & engine:

- ReAct data agent
- Domain models (step , trace, observation, result)
- Prompt & parser (ReAct parsing)
- Plot quality evaluator

Ports Layer (abstract interfaces, all ABCs)

- LLMPort
- Sandbox Port
- Storage Port

Fast API adapter -invokes-> Analyze Dataset use case

Analyze Dataset use case -invokes-> ReAct

Analyze Dataset use case -invokes-> Storage Port

ReAct -invokes-> LLMPort

ReAct -invokes->Sandbox Port

ReAct -invokes-> Models

ReAct -invokes-> Prompts

Open AI adapter -implements-> LLM Port

Mock LLM Adapter -implements-> LLM Port

Proceshags Sandbox Adapter -implements-> Sandbox Port

SQLite Session Repository -implements-> Storage Port

Directory structure

readme.docx

readme.pdf

Requirements.txt

Dockerfile

docker-compose.yml

.env.example

.env (original env with temporary keys)

.gitignore

day1 /

-run_day1.py (interactive and mock CLI)

data/

-ecommerce_sales_2024.csv (demo dataset with 500+ e-commerce orders)

- student_performance.csv (demo dataset with 150+ students)

examples/

-example_plot (plotly html example)

Src/

Chiron_core/

config.py (pydantic-settings)

Comain/models.py (step, observation, trace)

Ports/ (Abstract interfaces: LLMPort, SandboxPort, SessionStoragePort)

Adapters/ (LLM and Sanbox)

Engine/ (ReAct loop, prompt engineering, quality evaluator)

Use_cases/ (Application logic with AnalyzeDatasetUseCase)

utils/logger.py

tests/

conftest.py (test fixtures)

test_sandbox.py (timeout, network and filesystem)

test_react_agent.py (ReAct loop test and error recovery)

day2/

src/

chiron_service/ (FastAPI microservice)

config.py

main.py (Uvicorn entryptpoint)

storage/ (SQLite + SQLAlchemy models and repo)

api/ (router, routes, Pydantic schemas, factory app)

tests/

conftest.py (client test and SQLite fixtures)

test_api.py (9 HTTPs integration tests)

Reasoning Clarity

Core Engine

Native ReAct Loop (150ca. lines) made with Pydantic instead of LangChain (+CrewAI +AutoGen): generics frameworks introduce a lot of opaque abstraction levels, dependencies and a difficulty in tracking step-by-step reasoning. This native loop gives me full control, easy debug and deterministic tests.

Sandbox Execution

Isolated subprocess + TempDir + Patch socket runtime (alternative to Firecracker/dynamic docker-in-docker).

Startup time <100ms. Multilevel sandbox gives me filesystem isolation and native network blocking ('--network none').

Sandbox Strategy

“Sandbox Requirements (Minimum) — the sandbox must: execute code in a separate process, enforce an execution timeout, restrict filesystem access to a temporary working directory, prevent external network access”

Solution

- “subprocess.run” execute every code exectusions, invoking “sanbox_runner.py” a bootstrap dedicated runner
- “subprocess.run(....., timeout=15)” prevents blocked script or infinite loops, thanks to “TimeoutExpired” and returning error to the agent.
- “tempfile.TemporaryDirectory()” to save the dataset in a temp directory that is eliminated ath the run end, only Plotly .html are saved in a persistent directory “output/plots/”
- “sandbox_runner.py” kills in runtime low-layer networks of the “socket” module (“socket.socket”, “socket.create_connection”, “socket.getaddrinfo”) and proxy variables are emptied
- Garbage collector: “ProcessSandboxAdapter” applies and automatic retention manteining only last 5 geenerated charts.

Day2 solution: defense-in-depth garanted by flag “—network none” and non-root execution “appuser”

Persistence

SQLite + SQLAlchemy 2.0 repo pattern (alternative to PostgreSQL + Redis).

SQLite: no external database cluster necessity, but thanks to “SessionStoragePort” we can switch to PostgreSQL changing only DATABASE_URL.

LLM Integration

Standard protocol “/v1/chat/completions” (Gemini in my case from <https://generativelanguage.googleapis.com/v1beta/openai/>) , our LLMPort and OpenAILLMAdapter are universal and they meet the Hexagonal Architecture requirement.

Google-genai and boto3 SDK vendor: no proprietary SDKs that chain us to a single supplier or heavy gRPC dependences for our Docker image, like grpcio, protobuf or goole-auth-httpplib2. Standard adapter gives us the possibility to switch from Gemini to Ollama (local), Groq or OpenAI changing only 2 .env variables.

I choose Gemini 2.5 Flash Lite for these reasons:

- Ultra low latency: <2/3s per step
- Sufficient understanding of Pandas API and Plotly Express, with an well-done syntax.

- Big context window: more than 1M tokens, no memory overflows also with medium-big datasets and longer reasoning traces.
- No credit drain: I used an Free-Tier key, with a 15RPM rate-limit for development.

Software architecture

Hexagonal Architecture (Ports and adapter + use cases) & fat route handlers: `chiron_core` core doesn't import FastAPI nor SQLAlchemy. Orchestration is in "AnalyzeDatasetUseCase". No external dependences for our 21 complete test executed in less than 18s.

ReAct Loop native vs monolytic frameworks

Instead of using complex frameworks or black-boxes (ex. LangChain), I preferred implement a native ReAct loop:

- full prompt management control.
- faithful Reasoning trace saving and serialization (challenge requirement)
- self-healing: if the code produces an "KeyError" or a syntax error, the traceback is formatted with an observation to allow LLM to self-correct in the next step.

Multiple agent swarm

Observation traceback error direct injection instead of multiple agent swarm (Coder + critic+ reviewer). Our ReAct loop with traceback injection gives the LLM the possibility to correct syntax errors or "KeyError" in a single step in less than 2/3 seconds.

API contract

The endpoint `/analyze` supports custom datasets and pre-uploaded datasets. The default setup included 2 demo datasets:

- `ecommerce_sales_2024.csv` (more than 500 real ecommerce records, with

categories, discounts, rating, regions)
-“student_performance.csv” (150+ academic records)

Datastore choice

SQLite: no external dependences or complex configurations for local development and for the test suite. Direct support for ACID transactions, fast query on a single machine and natively integrated in Python.

Test Suite execution

To execute every automated test I used day1/tests and day2/tests, expecting this output (runned with “python -m pytest day1/tests day2/tests -v”):

day1/tests/test_react_agent.py::test_parse_react_response_execute_code PASSED

day1/tests/test_react_agent.py::test_parse_react_response_final_answer PASSED

day1/tests/test_react_agent.py::test_parse_react_response_fallback_resilience
PASSED

day1/tests/test_react_agent.py::test_react_agent_successful_flow PASSED

day1/tests/test_react_agent.py::test_react_agent_self_healing_error_recovery PASSED

day1/tests/test_react_agent.py::test_react_agent_max_iterations_cutoff PASSED

day1/tests/test_sandbox.py::test_sandbox_executes_valid_code PASSED

day1/tests/test_sandbox.py::test_sandbox_timeout_enforcement PASSED

day1/tests/test_sandbox.py::test_sandbox_network_blocking_socket PASSED

day1/tests/test_sandbox.py::test_sandbox_network_blocking_urllib PASSED

day1/tests/test_sandbox.py::test_sandbox_filesystem_isolation PASSED

day1/tests/test_sandbox.py::test_sandbox_plotly_artifact_extraction PASSED

day2/tests/test_api.py::test_health_endpoint PASSED

day2/tests/test_api.py::test_list_datasets_endpoint PASSED

```
day2/tests/test_api.py::test_analyze_endpoint_success PASSED
day2/tests/test_api.py::test_session_retrieval PASSED
day2/tests/test_api.py::test_multi_turn_session_persistence PASSED
day2/tests/test_api.py::test_session_not_found_returns_404 PASSED
day2/tests/test_api.py::test_empty_query_returns_422 PASSED
day2/tests/test_api.py::test_missing_dataset_returns_404 PASSED
day2/tests/test_api.py::test_plot_serving_security_traversal PASSED
```

```
===== 21 passed =====
```

Test Strategy for Non-deterministic components

As request in the 2nd Day requirements:

In our CI/CD tests suite the LLM responses are simulated by “MockLLMAdapter”, this guarantees 100% determinism, fast execution (<15s for 21 tests), no token usage, networks problems or rate-limiting

How to RUN

Day1 - Setting-up

```
python -m venv venv
```

```
venv\Scripts\activate #on windows
```

```
Source venv\bin\activate #on linux or macOS
```

```
Pip install -r requirements.txt
```

Day1 - Mock execution (without API credentials)

Python day1/run_day1.py --mock

Day1 - Real LLM execution

Python day1/run_day1.py --dataset day1/data/ecommerce_sales_2024.csv --query "Qual è la categoria di prodotti con il maggior fatturato? Crea un grafico interattivo a barre con Plotly."

Day2 – Uvicorn starting

python day2/src/chiron_service/main.py

The service will start at <http://localhost:8000>

Interactive documentation with Swagger UI: <http://localhost:8000/docs>

ReDoc: <http://localhost:8000/redoc>

Healthcheck: <http://localhost:8000/health>

Analysis: POST <http://localhost:8000/analyze>

Session history: GET <http://localhost:8000/sessions/{id}>

Chart Visualization: GET <http://localhost:8000/plots/{filename}>

Day2 – Docker starting

Docker compose up --build

The container will start isolated with a non-root user, mounting volumes for the persistency of: SQLite, datasets and generated charts.

Known limitations

Sandbox isolation at application level: without root privileges in windows and in linux the network blocking is made at Python runtime. So libraries written in C can elude our networking blocks without “seccomp” or “—network none” on Docker.

Our agent is optimized for CSV tabular datasets, so SQL databases, Parquets or excel are not implemented yet.

Every script generated for every step is an autonomous module, so we don't support complex multi-script workflows.

What I would improve with more time

- Converting agent prompts in DSPy modules with explicit signatures as `dataset_schema`, `question ->thought`, `python_code`.
- Make a quantitative evaluation metric for a LLM as judge .
- GEPA on a dev-set of 20/30 questions to define the best few-shot and instructions to maximize success percentage
- Adding a WebSocket in FastAPI to visualize in real time on client agent thoughts and coding lines as they are generated.
- Adding a static control of the code with `ast.parse` before execution to reject instantly dangerous calls to save a round-trip in our sandbox.
- Multiplot support for our agent to make combined plot or advanced HTML dashboards when a request needs complex comparisons.